VECTOR

FBLLib SecureBoot

Technical Reference

Flash Bootloader - SecureBoot Library vHsm

Version 1.0.0

Authors	Marco Riedl
Status	Released

Document Information

History

Author	Date	Version	Remarks
Marco Riedl	2019-12-13	1.0.0	Initial version

Reference Documents

No.	Source	Title	Version
[1]	Vector	Technical Reference – MICROSAR Memory Mapping	1.3.0
[2]	Vector	Technical Reference – MICROSAR CSM	
[3]	Vector	Technical Reference – VHSM CORE	
[4]	Vector	Technical Reference – Flash Bootloader OEM	
[5]	Vector	Technical Reference – Boot Manager	2.0.0
[6]	Vector	Reference Manual – HexView	



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6
1.1 Version 1.00.00 6

2 Introduction 7

3 Functional Description 8
3.1 Features 8
3.2 Limitations 8
3.3 Specific Features 8
3.3.1 Secure/Authenticated Boot 8
3.3.2 Secure Boot Update 8
3.3.3 Secure Boot Forced Sequential 8

4 Integration 9
4.1 Execution instance 9
4.2 vHsm Segment List 9
4.3 Scope of Delivery 9
4.3.1 FblSb 9

5 Configuration 11
5.1 FblSb 11
5.1.1 FblSb | FblSbGeneral 11
5.1.2 FblSb | FblSbMacOperations 13
5.2 Csm 14
5.2.1 Csm | CsmJobs 14
5.2.2 Csm | CsmKeys 14
5.3 Project Settings 15
5.3.1 Vector vHsm 15

6 API Description 16
6.1 Overview of FblSb Functions 16
6.2 Core functions 16
6.3 Callback functions 20

7 Glossary and Abbreviations 23
7.1 Glossary 23
7.2 Abbreviations 23

8 Contact 24

© 2020 Vector Informatik GmbH

Illustrations

Figure 5-1 DaVinci Configurator: FblSbGeneral Configuration 12
Figure 5-2 DaVinci Configurator: FblSbMacOperations Configuration 13
Figure 5-3 DaVinci Configurator: CsmJobs Configuration 14
Figure 5-4 DaVinci Configurator: CsmKeys Configuration 15
Figure 5-5 DaVinci Configurator: Add additional Module Definitions 15
Figure 5-6 DaVinci Configurator: Configure destination of the Pre-Config file 15

Tables

Table 4-1 Static files of FbSb 9
Table 4-2 Generated files of FbSb 9
Table 4-3 Template files of FbSb 10

1 Component History

The changes to this component are shown in the following.

1.1 Version 1.00.00

Initial version.

2 Introduction

This document covers the SecureBoot software component for vHsm use case. This component is used to update or verify different memory ranges with the the SecureBoot features of the vHsm.

3 Functional Description

The SecureBoot library abstracts the access to the hardware specific security module, here SHE.

3.1 Features

The SecureBoot library has the following general features:

- Using the vHsm Secure Boot Update (see [3]) mechanism to update the segment information in the vHsm.
- Using the vHsm Secure Boot (see [3]) mechanism to perform the verify the segments.
- Initialize the CMAC of the segments on the first startup.

3.2 Limitations

The SecureBoot library does not initialize the keys in the vHsm. For this purpose, the vHsm has its own feature to initialize specific key with random values on the first startup.

3.3 Specific Features

The following sections describe specific features of the SecureBoot library.

3.3.1 Secure/Authenticated Boot

The vHsm supports both boot modes (see [5] for more information):

- Secure Boot
- Authenticated Boot

The boot mode can initially be configured in the vHsm in the segment of a secure boot group.

Furthermore, during the SecureBootUpdate process the mode can be individually set with the vHsm Segmentation Table (see [3]).

3.3.2 Secure Boot Update

The Secure Boot Update process is required to update the segment information (address, length, mode and CMAC) of a specific group in the vHsm. This mechanism requires a structure which is defined in [3].

3.3.3 Secure Boot Forced Sequential

This feature is required in case initial values for SecureBoot segments were configured.

The Bootmanager would jump on the first startup to the Bootloader, which then initializes the segments in vHsm by triggering a “Secure Boot Forced Sequential”.

To use the this feature a “key” (flag) is required in the vHsm, see section 5.1.1.6. If an initial startup key is configured, it is also required to specify a Secure Boot Verify Forced Sequential CsmJob, which performs the necessary steps to initialize the segments.

4 Integration

Useful hints on the integration of the SecureBoot library can be found in the following sections.

4.1 Execution instance

The SecureBoot library needs to be added to each execution instance (Bootloader, Boot Manager, Updater) where it is necessary to verify or update the CMACs (SecureBoot segments).

4.2 vHsm Segment List

The vHsm requires a segment list to update internally the segment information.

It is possible to use HexView to create this list. Please refer to the [6] Reference Manual – HexView for more information.

4.3 Scope of Delivery

The delivery of the SecureBoot library contains the files which are described in the following sections.

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro. If the delivery is for GENy the generated files needs to be adapted manually (see chapter 5).

4.3.1 FblSb

4.3.1.1 Static Files

File Name	Description
fbl_secboot.c	This is the source file of the Secure Boot.
fbl_secboot.h	This is the header file of the Secure Boot.

Table 4-1 Static files of FbSb

4.3.1.2 Generated Files

File Name	Description
FblSb_Cfg.h	This is the configuration header file. It contains configuration switches.
FblSb_Lcfg.c	This is the configuration source file. It contains the configuration structures of the MacOperations.

Table 4-2 Generated files of FbSb

4.3.1.3 Template Files

File Name	Description
fbl_secboot_ap.c	This is the template source file of the Secure Boot. It contains adaptable functionality.

File Name	Description
fbl_secboot_ap.h	This is the template header file of the Secure Boot. It contains adaptable functionality.

Table 4-3 Template files of FbSb

5 Configuration

This chapter describes how the SecureBoot library is configured through the configuration tool DaVinci Configurator. A list of generated files and the respective description can be found in section 4.3).

To configure the SecureBoot library, a DaVinci configuration is needed. The bootloader delivery contains an example of such a configuration in the Demo folders, e.g. DemoBm, DemoFbl.

If the delivery supports GENy then the configuration is done manually by adaptation of template files located in BSW\FblSecBoot_Template.

The following sections provide a list of all configurable Boot Manager options, together with a short description.

The following modules are relevant to the configuration of the SecureBoot library:

- FblSb – secure boot
- Csm – AUTOSAR CSM (Crypto Security Manager)
- CryIf – AUTOSAR CRYIF (Crypto Interface)
- vHsm Driver

The following sections describes the DaVinci modules and the configuration aspects.



Note
There is no generator for GENy available. In this case the “generated files” described in section 4.3 need to be adapted manually. Template files are provided in BSW\FblSecBoot_Template.

5.1 FblSb

The following sections describe the configuration of the FblSb component.

5.1.1 FblSb | FblSbGeneral

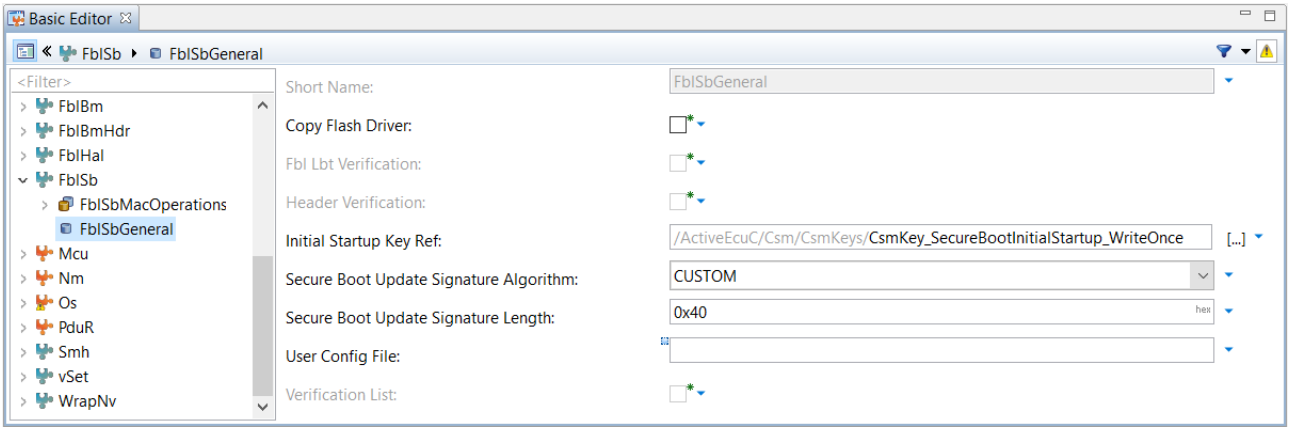


Figure 5-1 DaVinci Configurator: FblSbGeneral Configuration

5.1.1.1 Copy Flash Driver

This option enables an API to copy the flash driver from ROM to RAM. If the bootloader shall calculate the CMAC at production time, this option must be activated. This is necessary because the bootloader uses the flash driver to write the CMAC into the flash and the flash driver write function is only available when the flash driver is available in RAM.

This option is only relevant if the flash driver is downloaded by the diagnostic tool during the software download process. Otherwise, the normal flash driver initialization will copy the flash driver from ROM to RAM.

5.1.1.2 User Config File

Reference to an external text file that will be included during generation to “FblSb_Cfg.h”. The content of the user configuration file will be added at the end of the generated module configuration file and allows modification or extending the generated code.

5.1.1.3 Fbl Lbt Verification

This attribute specifies if the SecureBoot library supports the independent verification of the LogicalBlockTable of the Bootloader.

5.1.1.4 Header Verification

This attribute specifies if the SecureBoot library supports the independent verification of the BmHdrHeader.

5.1.1.5 Verification List

This attribute specifies if the BmHdrHeader should contain a verification list or the authentication header address.

5.1.1.6 Initial Startup Key Ref

This parameter refers to a CsmKey, which is used by the Bootmanager and Bootloader to determine if this is the initial/first startup of the ECU

The Bootmanager checks during startup if the key deviates from the initial value and depending on the result the Bootloader is started automatically. The Bootloader writes the key after the initial segments have been updated by the vHsm.



Note
This key needs to be configured as a WriteOnce key in the vHsm

5.1.1.7 Secure Boot Update Signature Algorithm

Specifies the algorithm used by the vHsm to determine the length of the signature used for the Secure Boot Update process. If the algorithm is not present, please select “CUSTOM”.

Note
This parameter is only used to automatically fill out the value in “Secure Boot Update Signature Length” in case of a known algorithm.

5.1.1.8 Secure Boot Update Signature Length

Length of the signature used by the vHsm during the Secure Boot Update sequence.

Note
This parameter is automatically filled out by the configuration of “Secure Boot Update Signature Algorithm”

5.1.2 FblSb | FblSbMacOperations

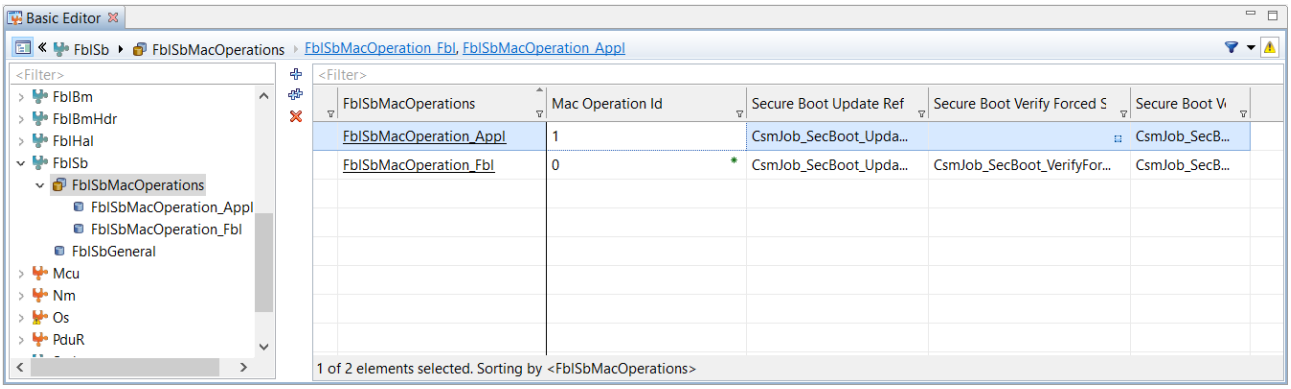


Figure 5-2 DaVinci Configurator: FblSbMacOperations Configuration

5.1.2.1 Secure Boot Update Ref

This parameter references the corresponding CsmJob to trigger the Secure Boot Update mechanism on the vHsm, see section 5.2.1

Note
Adding at least one Secure Boot Update Reference will enable the switch FBLSB_ENABLE_GENERATE_CMAC.

5.1.2.2 Secure Boot Verify Ref

This parameter references the corresponding CsmJob to trigger the verification of all segments of a specific SecureBoot group in the vHsm during the Boot Manager, see section 5.2.1.

5.1.2.3 Secure Boot Verify Forced Sequential Ref

This parameter references the corresponding CsmJob to trigger the initialization of the configured SecureBoot segments, see section 5.2.1.

5.2 Csm

For more details about the CSM, please refer to [2] Technical Reference – MICROSAR CSM. The following sections show an example configuration but the detailed description is not considered in this document.

5.2.1 Csm | CsmJobs

Figure 5-3 shows an example configuration of the CsmJobs for a SHE based configuration using the CryptoWrapper module.

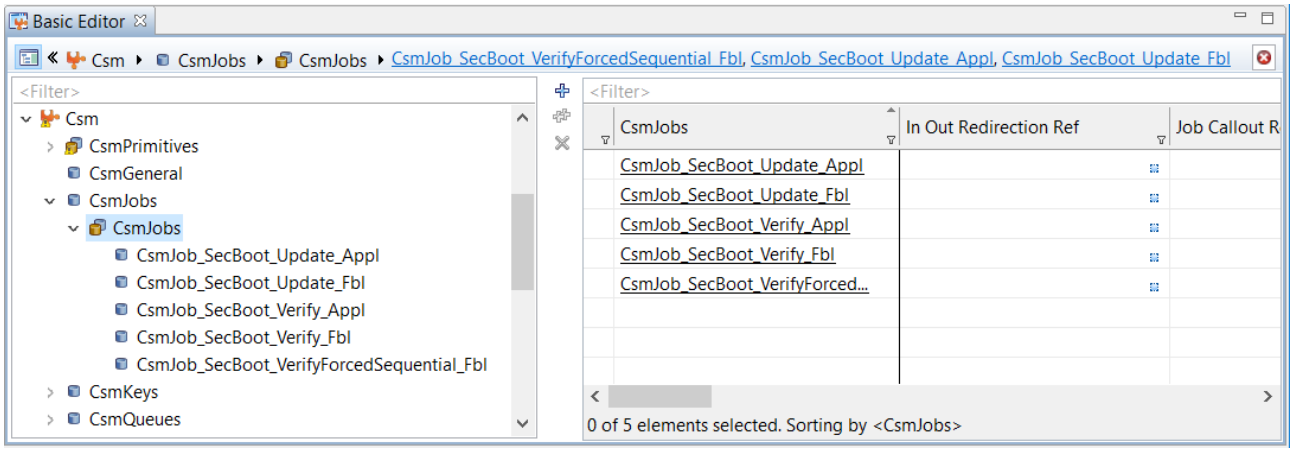


Figure 5-3 DaVinci Configurator: CsmJobs Configuration

5.2.2 Csm | CsmKeys

Figure 5-4 shows an example configuration of the CsmKeys for a SHE based configuration using the CryptoWrapper module.

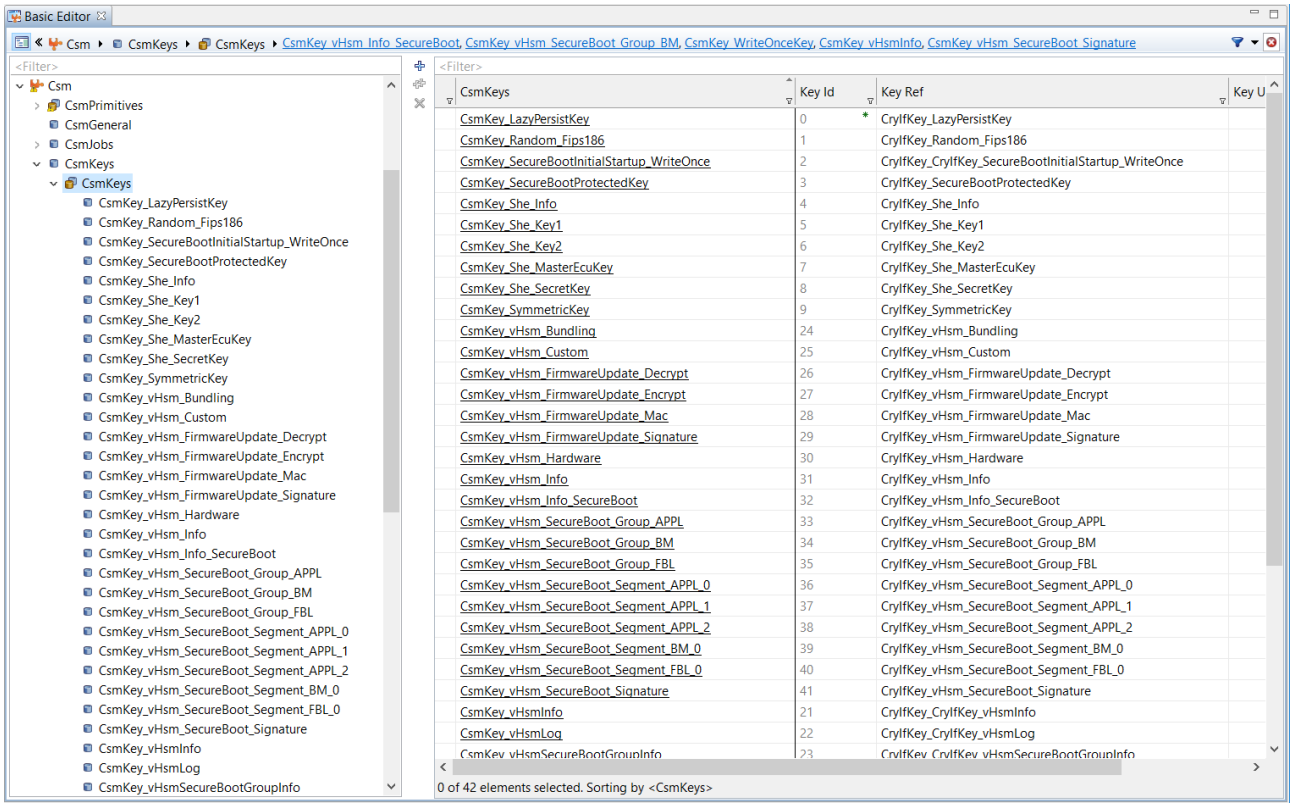


Figure 5-4 DaVinci Configurator: CsmKeys Configuration

5.3 Project Settings

There are further project specific setting depending on the used crypto driver/module.

5.3.1 Vector vHsm

In case the Vector vHsm solution is used, the vHsm Pre-Config file needs to be referenced by the host configuration (e.g. Boot Manager, Flash Bootloader). After the reference has been added the DaVinci Configurator needs to be restarted to apply the changes.

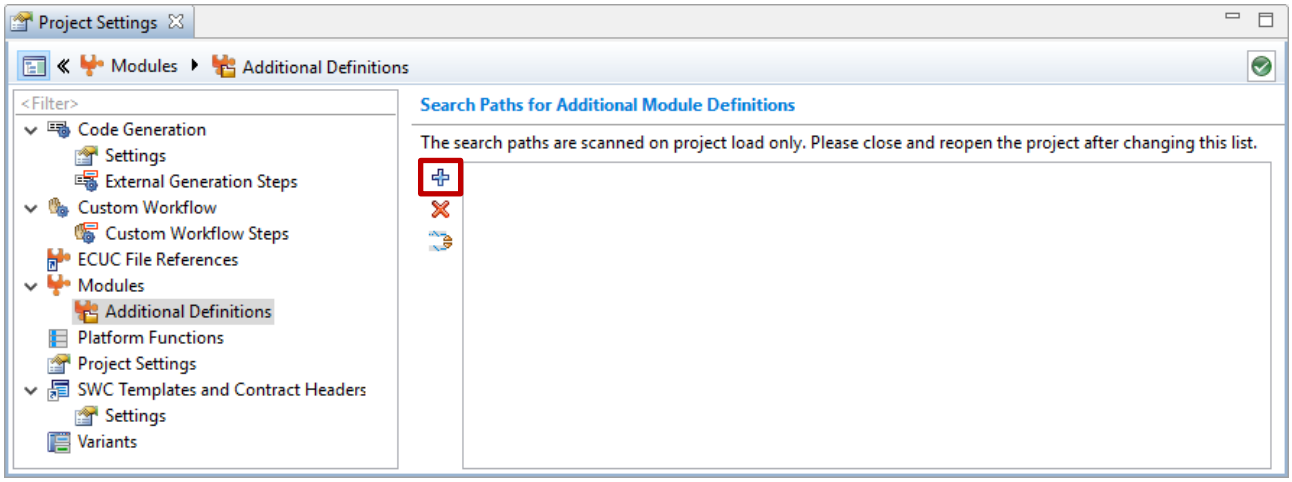


Figure 5-5 DaVinci Configurator: Add additional Module Definitions

The location of the Pre-Config file can be configured within the vHsm configuration.

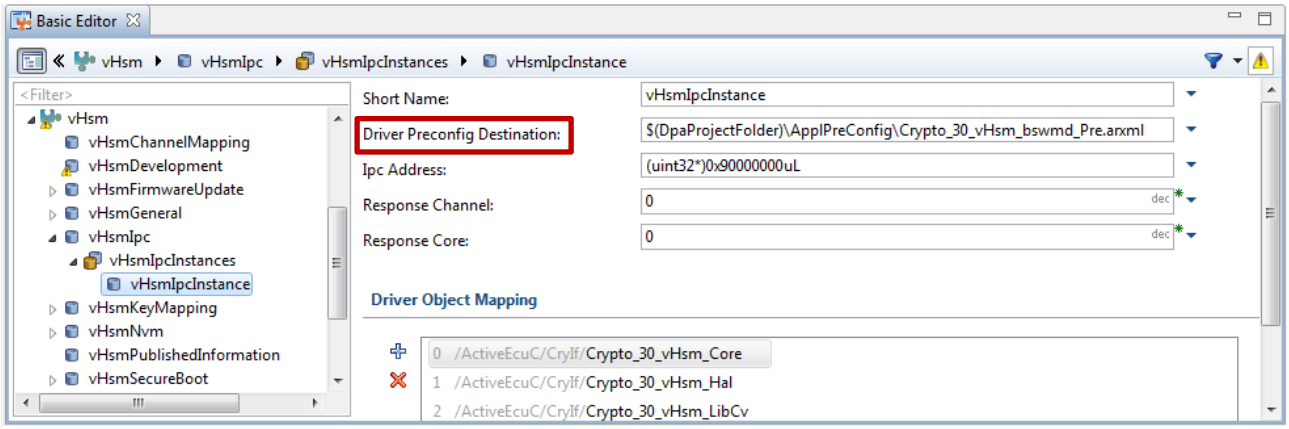


Figure 5-6 DaVinci Configurator: Configure destination of the Pre-Config file

6 API Description

The API of the SecureBoot library is described in this chapter.

6.1 Overview of FblSb Functions

The next section provides the API of the FblSb module.

6.2 Core functions

The next sections contains a list of the core functions of the FblSb module.

6.2.1.1 FblSb_Init

Prototype	
`Void FblSb_Init ( void )`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Initializes the crypto stack, e.g. Csm, CryIf, Crypto	
Particularities and Limitations	
-	
Call context	
-	

6.2.1.2 FblSb_VerifyFblLbt

Prototype	
`tFblResult FblSb_VerifyFblLbt ( const V_MEMRAM1 tFblHeader V_MEMRAM2 V_MEMRAM3 * fblHeader )`	
Parameter	
`fblHeader [in]`	Pointer to the FBL Header structure
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed

Functional Description
Verifies the MAC of the LogicalBlockTable of the Bootloader
Particularities and Limitations
> This function will always return kFblFailed
Call context
-

6.2.1.3 FblSb_VerifyHeader

Prototype	

`tFblResult` **`FblSb_VerifyHeader`**
`(`
`  const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHdrHeader`
`)`	

Parameter	
`bmHdrHeader [in]`	Pointer to the FblBmHdrHeader structure
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed
Functional Description	
Verifies the MAC of the BmHdrHeader structure	
Particularities and Limitations	
> This function will always return kFblFailed	
Call context	
-	

6.2.1.4 FblSb_VerifySegments

Prototype	

`tFblResult` **`FblSb_VerifySegments`**
`(`
`  const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHdrHeader`
`)`	

Parameter	
`bmHdrHeader [in]`	Pointer to the FblBmHdrHeader structure
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed

Functional Description
Verifies all segments of a SecureBoot group in the vHsm
Particularities and Limitations
-
Call context
-

6.2.1.5 FblSb_UpdateFblMac

Prototype	

`tFblResult` **`FblSb_UpdateFblMac`**
`(`
`  V_MEMROM1 tFblHeader V_MEMROM2 V_MEMROM3 * fblHeader`
`)`	

Parameter	
`fblHeader [in]`	Pointer to the FBL Header
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed
Functional Description	
Generate and write the necessary MACs for the FBL	
Particularities and Limitations	
> This function is only available if the Secure Boot Update operation is configured.	
Call context	
-	

6.2.1.6 FblSb_UpdateBlockMac

Prototype	

`tFblResult` **`FblSb_UpdateBlockMac`**
`(`
`  const V_MEMRAM1 tBlockDescriptor V_MEMRAM2 V_MEMRAM3 * blockDescriptor,`
`  V_MEMRAM1 FL_SegmentListType V_MEMRAM2 V_MEMRAM3 * segmentList`
`)`	

Parameter	
`blockDescriptor [in]`	Pointer to the logical block
`segmentList [in]`	Pointer to the segment list of the downloaded data
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed

Functional Description
Generate and write the necessary MACs for a logical block
Particularities and Limitations
> This function is only available if the Secure Boot Update operation is configured.
Call context
-

6.2.1.7 FblSb_UpdateInitialSegments

Prototype	

`tFblResult` **`FblSb_UpdateInitialSegments`**
`(`
`  Void`
`)`	

Parameter	
-	-
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed
Functional Description	
Triggers the Secure Boot Verify Forced Sequential on the vHsm to initialize the CMACs based on the initial values of each segment.	
Particularities and Limitations	
-	
Call context	
-	

6.2.1.8 FblSb_CopyFlashDriver

Prototype	

`void` **`FblSb_CopyFlashDriver`**
`(`
`  Void`
`)`	

Parameter	
-	-
Return code	
-	-
Functional Description	
Copies the flash driver (located in BSW\Flash) into RAM.	

Particularities and Limitations
> This function is only available if Copy Flash Driver is enabled.
Call context
-

6.3 Callback functions

The next sections contains a list of callback functions which are used by the core functions or by the Boot Manager/Bootloader directly.

6.3.1.1 ApplFblSbStartup

Prototype	
`void ApplFblSbStartup ( void )`	
Parameter	
–	-
Return code	
–	-
Functional Description	
Secure boot initialization callback. This might include > Initializes the CMACs (see section 6.2.1.7)	
Particularities and Limitations	
-	
Call context	
> This function is called during Bootloader initialization to setup secure boot	

6.3.1.2 ApplFblSbIsInitialStartup

Prototype	
`tFblResult ApplFblSbIsInitialStartup ( void )`	
Parameter	
–	-
Return code	
`kFblOk`	This is the initial/first startup
`kFblFailed`	This is not the initial/first startup

Functional Description
This function checks via the Initial Startup Key, if this is the first startup or not.
Particularities and Limitations
-
Call context
> This function is called during Bootmanager startup

6.3.1.3 ApplFblSbClrInitialStartup

Prototype	
`void ApplFblSbClrInitialStartup ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
This function clears the initial startup condition. The default implementation writes a specific value to the Initial Startup Key.	
Particularities and Limitations	
-	
Call context	
> This function is called after the Bootloader as triggered the initialization of the segments (see section 3.3.3)	

6.3.1.4 ApplFblSbGetSignatureInfo

Prototype	
`void ApplFblSbGetSignatureInfo ( const V_MEMRAM1 tBlockDescriptor V_MEMRAM2 V_MEMRAM3 * blockDescriptor, const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHdrHeader, V_MEMRAM1 uint32 V_MEMRAM2 V_MEMRAM3 * signatureAddress, V_MEMRAM1 uint32 V_MEMRAM2 V_MEMRAM3 * signatureLength )`	
Parameter	
`blockDescriptor`	Pointer to the block descriptor
`bmHdrHeader`	Pointer to the FblBmHdrHeader structure

`signatureAddress`	Holds a pointer to the memory location in which the address of the signature is stored
`signatureLength`	Holds a pointer to the memory location in which the length of the signature is stored
Return code	
–	-
Functional Description	
This function is used to retrieve the address location and the length of the signature. The default implementation assumes that the signature is placed directly in front of the segment list.	
Particularities and Limitations	
-	
Call context	
> This function is called after the Bootloader as triggered the initialization of the segments (see section 3.3.3)	

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Secure Boot	Integrity check at boot time by means of an HTA.
Hardware Trust Anchor	Hardware-based authoritative entity for which trust is assumed and not derived. This can be implemented by means of an HSM or OTP memory.
Flash Bootloader	Software stack to implement a software update capability, e.g. by means of UDS protocol. The Flash Bootloader is typically executed in a separate context.
Flash Bootloader Updater	Application like software to update an existing Flash Bootloader.
Presence Pattern	Store validity information (block and/or application validity) in flash memory at the respective logical block in case no EEPROM or FEE is available.
Logical Block	Individually erasable/writable software download entity.

7.2 Abbreviations

Abbreviation	Description
CMAC	Block Cipher-based Message Authentication Code
CSM	AUTOSAR Crypto Service Manager
FBL	Flash Bootloader
HSM	Hardware Security Module Dedicated microcontroller that safeguards and manages cryptographic keys and secure memory. The HSM creates a secure execution environment for implementing cryptographic primitives.
HTA	Hardware Trust Anchor
LBT	Logical Block Table
vHsm	Vector HSM-based HTA solution

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com